MVP PRD — Self-Hosted Stock & ETF Research Platform

> Safety and product positioning: this platform must be described as a research and decision-support system, not as a source of guaranteed returns or personalized financial advice. The MVP should produce evidence-backed research signals, paper-trading simulations, risk scenarios, and explicit “no signal” outcomes. Do not implement autonomous live order execution in the MVP. VectorBT itself warns that its software is educational and that users remain responsible for trading outcomes; NautilusTrader similarly separates software functionality from advisory services and user compliance obligations.

Core principle

Quantitative code calculates every number and signal. The local LLM retrieves evidence and explains the result; it never invents prices, indicators, forecasts, or trades.

MVP operating mode

Historical research, screening, alerts, portfolio analytics and paper trading only. Live execution is deferred until independent validation and compliance review.

Trust model

Every displayed recommendation must be reproducible from a versioned data snapshot, strategy version, configuration, model version and audit trail.

---

1. Product definition

1.1 Objective

Build a modular, self-hosted web platform that:

- Ingests and persists stock, ETF, benchmark, fundamental and corporate-action data.
- Supports **long-term**, **swing**, and **intraday research**.
- Generates deterministic entry, exit, hold and abstain signals.
- Runs reproducible backtests and paper portfolios.
- Displays risk, uncertainty, assumptions and invalidation conditions.
- Uses a locally hosted LLM only for orchestration, retrieval and natural-language explanation.
- Preserves raw data, derived values, model outputs, user-visible rationales and audit history.

Internal investment-platform capability models also emphasize security masters, market-data governance, corporate actions, performance, risk, reconciliation, audit trails and reporting as separate capabilities, which supports keeping these concerns modular rather than building one monolith.

1.2 Personas

1. **Retail investor:** watchlists, understandable signal cards, portfolio risk and scheduled reports.
2. **Advanced investor:** custom screens, parameterized strategies, detailed backtests and factor analysis.
3. **Quant researcher:** notebooks/SDK, feature sets, experiment tracking, walk-forward tests and model registry.
4. **Administrator:** data-source configuration, user permissions, audit logs, backups and system health.

1.3 Primary journeys

- Search an instrument → inspect chart, fundamentals, trend, risk and latest evidence.
- Add instruments to a watchlist → receive alerts when a validated rule changes state.
- Run a scanner → filter by liquidity, trend, valuation, volatility and signal confidence.
- Open a signal → see trigger, entry zone, stop/invalidation, target methodology, evidence, risks and backtest context.
- Create/import a paper portfolio → view allocation, exposure, drawdown, attribution and rebalance suggestions.
- Configure a strategy → backtest → validate out-of-sample → promote to paper trading.
- Ask the local assistant “Why is this signal active?” → receive a sourced explanation generated from stored calculations.

1.4 Proposed MVP success criteria

These are **acceptance targets**, not market-performance promises:

- Every signal can be reproduced using a stored data_snapshot_id and strategy_version_id.
- No signal is emitted when required data-quality checks fail.
- 100% of signal cards include timestamp, horizon, confidence, risk limits, evidence and invalidation rule.
- Backtests account for transaction costs, slippage, market calendars and corporate actions.
- API p95 response under 2 seconds for cached instrument pages and under 5 seconds for normal scans on the MVP universe.

- Scheduled ingestion reports completeness, staleness, duplicate and adjustment status.
- Local reasoning evaluation shows no unsupported numeric claims in the release test set.
- System restores successfully from an automated backup in a documented recovery test.

---

2. Scope boundaries

MVP includes

- Stocks, ETFs and benchmarks; configurable initial universe.
- Daily bars for all use cases; optional 1/5/15/60-minute bars where licensed data exists.
- CSV/Parquet import plus pluggable data-provider adapters.
- SEC filing/fundamental ingestion for US instruments.
- Technical, fundamental, factor, portfolio and regime analysis.
- Watchlists, scanner, paper portfolios, alerts and HTML/PDF/CSV reports.
- Rule-based strategies and selected classical ML baselines.
- Vectorized and event-driven backtesting.
- Local LLM explanation and retrieval.
- Docker Compose deployment on one Linux host.

Explicitly excluded from MVP

- Automatic broker order routing, custody, deposits or withdrawals.
- Options/futures/crypto execution.
- Social-copy trading or guaranteed-price predictions.
- Reinforcement-learning trading agents.
- Tick/order-book strategies requiring colocation or ultra-low latency.
- Training a foundation model from scratch.
- Unrestricted code execution by the LLM.
- Tax advice or personalized suitability decisions.

---

3. Recommended architecture

Use a **modular monolith plus workers** initially: one FastAPI application with clear domain packages, separate Prefect workers, local model server and shared storage. This is materially easier to test and operate than premature microservices, while service interfaces allow later extraction. Prefect supports Python-native scheduling, retries, state tracking and

portable execution; TimescaleDB adds time partitioning, continuous aggregates, compression and retention while retaining PostgreSQL compatibility.

The architecture below intentionally separates deterministic analytics from language generation. This reflects Qwen's own documentation that LLM function calls can be malformed and require application-side validation, while vLLM's schema-constrained modes can guarantee parsable JSON structure without guaranteeing the quality of the requested action.

Figure 1: Self-hosted MVP architecture separating deterministic market-data and risk engines from the local LLM explanation layer, with immutable evidence and audit trails.

Technology baseline

Area	MVP recommendation	Alternatives / decision
Frontend	Next.js/React, TypeScript, Lightweight Charts or Plotly	Streamlit only for an engineering prototype
API	FastAPI, Pydantic, SQLAlchemy, Alembic	Django if admin-heavy
Jobs	Self-hosted Prefect	Celery/RQ for simpler queues
Primary DB	PostgreSQL + TimescaleDB	Plain PostgreSQL for very small datasets
Archive	Versioned local/S3-compatible Parquet via MinIO	Filesystem for single-user development
Cache	Redis, optional in first release	PostgreSQL cache table
Analytics	pandas/Polars, NumPy, SciPy, statsmodels	DuckDB for ad-hoc Parquet queries
Indicators	TA-Lib primary; custom verified indicators	pandas-ta-classic as convenience
Backtesting	VectorBT research + NautilusTrader confirmation	backtesting.py for teaching/simple strategies
Portfolio	PyPortfolioOpt + QuantStats	skfolio/Riskfolio-Lib in V2
ML	scikit-learn, LightGBM/XGBoost, statsmodels	Qlib in V2
Forecasting	sktime or Darts	Begin with statistical baselines
Experiments	MLflow	Qlib Recorder if Qlib becomes central
Observability	OpenTelemetry, Prometheus, Grafana	Loki for centralized logs
Local LLM	Ollama development; vLLM production GPU	llama.cpp for CPU/GGUF portability
Retrieval	PostgreSQL full-text + pgvector	Separate vector DB only after scale requires it

VectorBT is optimized for vectorized parameter sweeps on pandas/NumPy with Numba/Rust acceleration, while NautilusTrader provides deterministic event-driven simulation with configurable fills, fees, latency and shared research/live semantics.

PyPortfolioOpt supplies efficient-frontier, Black-Litterman, shrinkage and hierarchical-risk-parity methods; QuantStats provides performance metrics, drawdowns and tear sheets.

Do not add Feast to the first MVP. Use versioned feature tables and views in PostgreSQL. Add Feast in V2 only if online/offline feature consistency becomes operationally difficult; Feast is expressly designed to manage and serve the same feature definitions during training and inference.

---

4. Data strategy

4.1 Source policy

Create a MarketDataProvider interface:

```
class MarketDataProvider(Protocol):
    def instruments(self) -> list[InstrumentRecord]: ...
    def bars(self, request: BarsRequest) -> Iterable[Bar]: ...
    def corporate_actions(self, request: ActionRequest) -> Iterable[CorporateAction]: ...
    def fundamentals(self, request: FundamentalRequest) -> Iterable[FundamentalFact]: ...
    def health(self) -> ProviderHealth: ...
```

Adapters must support licensed vendors, user-uploaded CSV/Parquet and public regulatory information. SEC exposes EDGAR data through public APIs subject to its automated-access policies; NYSE corporate-action products include splits, dividends, distributions, listings, suspensions and delistings. Free convenience feeds may be enabled only for development: Qlib explicitly says its Yahoo-derived data may be imperfect and recommends users prepare higher-quality data.

4.2 Canonical bar model

Store:

- instrument_id, venue_id, timeframe
- ts_open, ts_close, UTC timezone
- raw open/high/low/close/volume/vwap/trade_count
- adjusted values and adjustment_factor
- currency, source, ingestion timestamp
- vendor revision, content hash and quality status

Primary key: (instrument_id, timeframe, ts_open, source_revision). Maintain a current view, but never overwrite historical vendor revisions.

4.3 Data lifecycle

1. Land immutable raw payload/Parquet and calculate checksum.
2. Validate schema, timezone, price relationships, duplicates and monotonic timestamps.
3. Normalize identifiers, currency, calendar and units.
4. Reconcile overlaps against a secondary source when available.
5. Apply split/dividend adjustments through versioned corporate-action records.
6. Publish a data_snapshot.
7. Compute indicators/features against that snapshot.
8. Archive raw data indefinitely; compress mature time-series partitions.
9. Back up PostgreSQL and object storage independently.

Historical ETF requirements should preserve security-master history, NAV/price history, corporate actions, index/model changes, stale-price exceptions, data lineage and backups rather than merely storing adjusted closing prices.

---

5. Signal engine

5.1 Standard output contract

```
{
  "signal_id": "uuid",
  "instrument_id": "uuid",
  "as_of": "RFC3339",
  "horizon": "LONG_TERM|SWING|INTRADAY",
  "state": "ENTER_LONG|EXIT|REDUCE|HOLD|WATCH|NO_SIGNAL",
  "entry_zone": {"low": 0, "high": 0},
  "invalidation": {"rule": "...", "level": 0},
  "target_method":
"risk_multiple|volatility_band|fundamental_value",
   "position_risk": {"max_loss_pct": 0, "suggested_size_pct":
0},
  "confidence": 0,
  "quality_gate": "PASS|WARN|FAIL",
  "strategy_version_id": "uuid",
  "data_snapshot_id": "uuid",
  "evidence_ids": [],
  "reason_codes": [],
  "limitations": []
}
```

The platform must permit NO_SIGNAL; confidence must never be shown as “probability of profit” unless it has been explicitly calibrated and validated as such.

5.2 MVP strategy families

Horizon	Strategies
Long-term	quality/value composite, earnings and cash-flow trend, valuation versus history/sector, 200-day trend filter, ETF expense/liquidity/tracking review, periodic rebalance
Swing	moving-average breakout, momentum continuation, RSI/Bollinger mean reversion, volatility contraction, gap/event follow-through
Intraday research	opening-range breakout, VWAP reclaim/rejection, intraday momentum, volatility expansion; paper only

Every strategy implements:

```
class Strategy:
    metadata: StrategyCard
    def required_features(self) -> list[FeatureSpec]: ...
    def generate(self, context: MarketContext) ->
RawSignal: ...
    def risk_plan(self, signal: RawSignal, portfolio:
PortfolioState) -> RiskPlan: ...
```

5.3 Risk gate

Before publishing a signal:

- Data freshness/completeness passes.
- Instrument is tradable and not suspended/delisted.
- Liquidity and spread proxies pass.
- Portfolio concentration, sector and correlated exposure limits pass.
- Volatility-based size and maximum-loss budget are calculated.
- Earnings/dividend/corporate-action proximity is disclosed.
- Stop or invalidation level is defined.
- Conflicting strategies reduce confidence or produce WATCH.
- Kill switch suppresses outputs during quality incidents.

5.4 Confidence methodology

Calculate confidence deterministically from:

- strategy validation score;
- current regime similarity to validated periods;
- feature completeness;
- agreement among independent signals;
- liquidity;

- model calibration;
- sensitivity to parameter changes.

Store component scores separately. Apply caps for stale data, unseen regimes or unstable parameters. The LLM may explain these components but cannot modify them.

---

6. Backtesting and model validation

Each backtest must record universe membership, survivorship policy, data snapshot, corporate-action version, calendar, fill timing, commission, spread/slippage assumptions, position limits and random seed.

Required validation:

- chronological train/validation/test splits;
- walk-forward evaluation;
- no same-bar close signal and close fill unless explicitly modeled;
- benchmark and buy-and-hold comparison;
- parameter-sensitivity heatmaps;
- delayed-entry and higher-cost stress tests;
- subperiod, volatility-regime and sector breakdowns;
- bootstrap/Monte Carlo uncertainty where appropriate;
- paper-trading promotion gate.

Minimum reported metrics: total/annualized return, volatility, Sharpe/Sortino with assumptions, maximum drawdown, Calmar, win rate, payoff, exposure, turnover, costs, trade count and benchmark-relative results. Qlib offers a broader ML research pipeline spanning data processing, modeling, backtesting and portfolio analysis, but should be evaluated in V2 rather than made an MVP dependency.

---

7. Local reasoning and explainability

Recommended design

1. User asks a question.
2. Intent router selects allowlisted read-only tools.
3. Tools retrieve stored bars, computed indicators, signal components, backtest summaries, filings and strategy cards.
4. Evidence compiler creates a compact, signed context package.
5. Local LLM returns JSON matching ExplanationSchema.
6. Validator rejects unknown evidence IDs, unsupported numbers or missing caveats.
7. Renderer produces the readable explanation.

8. Store prompt template version, tool calls, evidence IDs, final answer and validation result.

Candidate models and runtimes

- **Qwen3 8B:** practical first candidate; supports thinking/non-thinking modes and documented Hermes-style function calling through vLLM. The project warns that protocol adherence is not guaranteed, so tool arguments still require validation.
- **gpt-oss-20b:** stronger workstation candidate for reasoning/tool use; Apache-2.0, designed for local/specialized use and documented to run within approximately 16 GB using its supplied quantization.
- **gpt-oss-120b:** evaluation option for a dedicated high-memory GPU host, not the default MVP deployment.
- **Runtime:** Ollama for development and single-user setups; vLLM for authenticated, concurrent GPU serving. Both support tool calling; vLLM offers schema-constrained tool arguments in named/required modes.

Do **not** store or expose unrestricted hidden chain-of-thought. Persist a concise **decision trace**: retrieved evidence, tool inputs/outputs, reason codes, validation results and final user-facing rationale. Even gpt-oss documentation says raw reasoning is not intended for end-user display.

---

8. Core schema

Entity	Essential fields
instrument	identifiers, symbol history, type, venue, currency, status
corporate_action	type, ex/effective dates, factor/cash amount, revision
market_bar	OHLCV/VWAP, timeframe, source, revision, quality
fundamental_fact	taxonomy, period, filed date, value, unit, accession
data_snapshot	source versions, cutoff timestamp, hashes
feature_definition	logic, parameters, code hash, version
feature_value	snapshot, instrument, timestamp, value, quality
strategy_version	code hash, config, horizon, owner, status
signal	state, confidence, entry/invalidation/risk, reason codes
backtest_run	snapshot, strategy version, assumptions, metrics
model_version	artifact URI, training window, features, metrics
portfolio/account	base currency, risk policy, paper/live flag
transaction/position	paper fills, lots, fees, realized/unrealized P&L
watchlist/alert_rule	owner, conditions, channels, cooldown
evidence_item	source, excerpt/structured fact, timestamp, hash
reasoning_run	model/template versions, tools, evidence, final output
audit_event	actor, action, entity, before/after hashes, trace ID
job_run/data_issue	status, retries, row counts, exception details

Use UUIDs, UTC timestamps, JSONB only for extensible payloads, foreign keys for lineage, and append-only audit/revision tables.

---

9. APIs and jobs

REST endpoints

- /instruments, /instruments/{id}/bars|fundamentals|analysis
- /watchlists, /scanner/runs
- /strategies, /strategies/{id}/backtests
- /signals, /signals/{id}/explanation
- /portfolios, /portfolios/{id}/analytics|rebalance
- /alerts, /reports
- /assistant/query
- /admin/providers|jobs|quality|models|audit
- /health/live, /health/ready, /metrics

Use OpenAPI-generated clients, cursor pagination, idempotency keys for mutations and role-based authorization.

Scheduled jobs

- instrument/security-master refresh;
- daily/intraday bar ingestion;
- corporate-action and filing update;
- reconciliation and quality checks;
- indicator/feature materialization;
- scanner and signal generation;
- alert evaluation;
- portfolio mark-to-market;
- backtest/model evaluation;
- report generation;
- compression, retention, backup and restore verification.

---

10. UX requirements

1. **Home:** market status, portfolio risk, active alerts, stale-data banner.
2. **Instrument workspace:** chart, timeframes, indicators, fundamentals, events, signal history and evidence.
3. **Signal card:** action state, horizon, confidence components, entry zone, invalidation, scenario outcomes and “Why?”

4. **Scanner:** saved filters, sortable results, freshness and strategy badges.
5. **Backtest lab:** assumptions, equity/drawdown curves, trades, costs, benchmark, walk-forward and robustness views.
6. **Portfolio:** allocation, concentration, sector/country/currency exposure, correlation, attribution and rebalance preview.
7. **Operations:** ingestion status, freshness heatmap, exceptions, model versions and audit search.

The internal Data Analysis Platform - Overview.docx similarly uses visible processing stages, validation results, activity logs and status tracking; those patterns are useful for the platform's operator experience.

---

11. Observability, security and NFRs

- Structured JSON logs with trace_id, snapshot, strategy and model versions.
- OpenTelemetry traces for API, job, DB and LLM tool calls; it is a vendor-neutral framework for traces, metrics and logs.
- Prometheus/Grafana dashboards; MLflow for experiment/model tracking and LLM evaluation.
- Metrics: ingestion lag, missing bars, revisions, job failures, signal counts, abstentions, drift, backtest duration, API latency and LLM citation failures.
- Argon2 password hashing or OIDC, RBAC, rate limits, CSRF protection, TLS, encrypted backups and secret files outside source control.
- LLM tools are allowlisted, read-only by default, schema validated and executed by the application—not by the model.
- No shell, SQL, network or filesystem tool is exposed directly to user prompts.
- Pin dependencies; generate SBOM; scan containers and secrets in CI.
- Single-node MVP must support thousands of instruments and years of bar history; scale workers, archive and database independently later.
- Budget controls: batch indicator computation, compressed history, model unload/idle policy, quantized local models and cached explanations.

---

12. Repository contract for the coding agent

```
market-platform/
  apps/
    api/                # FastAPI routes and dependency injection
    web/                # Next.js frontend
```

```
    worker/              # Prefect deployments
  packages/
    domain/              # entities, enums, pure business rules
    data/                # providers, normalization, quality, adjustments
    features/            # indicators and point-in-time feature pipeline
    strategies/          # registry and horizon-specific strategies
    risk/                # sizing, limits, portfolio risk
    backtest/            # vectorized and event-driven adapters
    portfolio/           # accounting, optimization, attribution
    ml/                  # datasets, training, calibration, registry
    reasoning/           # tools, RAG, evidence and output validation
    alerts/              # rule evaluation and channels
    reporting/
    observability/
  db/
    migrations/
    seeds/
  tests/
    unit/
    integration/
    property/
    golden/
    e2e/
    performance/
  infra/
    compose/
    grafana/
    prometheus/
    otel/
  docs/
    adr/
    strategy-cards/
    data-contracts/
    runbooks/
```

```
    pyproject.toml
    docker-compose.yml
    .env.example
    Makefile
```

Coding rules

- Domain calculations are pure functions where possible.
- Every strategy, feature and model is versioned and immutable after promotion.
- Decimal types for monetary values; never binary float for portfolio accounting.
- No future data may enter a feature calculation.
- API schemas and DB migrations are reviewed together.
- Each module exposes interfaces; vendor-specific code stays in adapters.
- All generated explanations must reference valid evidence IDs.

---

13. Delivery roadmap

MVP

1. Repository, authentication, Docker Compose and schema.
2. CSV/Parquet provider, one development feed, SEC connector and quality pipeline.
3. Charts, watchlists and canonical indicators.
4. Strategy registry, risk gate and signal cards.
5. VectorBT backtests plus event-driven confirmation for promoted strategies.
6. Paper portfolios, scanner, alerts and reports.
7. Local LLM evidence workflow.
8. Observability, backups, security hardening and release tests.

V2

- Additional licensed feeds and exchanges.
- Qlib/Feast evaluation, model ensembles and calibrated ML signals.
- Intraday WebSocket ingestion, richer event detection and news-document ingestion.
- ETF holdings, NAV/premium-discount and tracking analysis.
- Multi-user organizations, SSO and stronger governance.
- Mobile-responsive alert center.

Future

- Broker sandbox integration followed by human-confirmed execution only after legal, security and model-risk review.
- Advanced optimization, options analytics and alternative data.
- Distributed workers/Kubernetes.

- Strategy marketplace with signed packages and approval workflow.

---

14. Definition of done

The AI coding agent must not mark the MVP complete until:

- Fresh deployment succeeds from documented commands.
- Unit, integration, property, golden backtest and Playwright E2E tests pass.
- Synthetic split/dividend/delisting fixtures prove adjustment correctness.
- Look-ahead tests fail intentionally contaminated strategies.
- A backtest rerun produces identical outputs from the same snapshot and seed.
- Signal suppression is verified for stale, missing and conflicting data.
- LLM red-team prompts cannot invoke unapproved tools or alter a signal.
- Explanation golden tests contain no unsupported numbers.
- Dependency, container, secret and license scans pass.
- Backup restoration, migration rollback and incident runbooks are exercised.
- Demo data clearly labels itself as non-production and paper-only.